Fundamental Programming in JAVA

ACCESS SPECIFIERS (VISIBILITY MODIFIERS)

Objectives

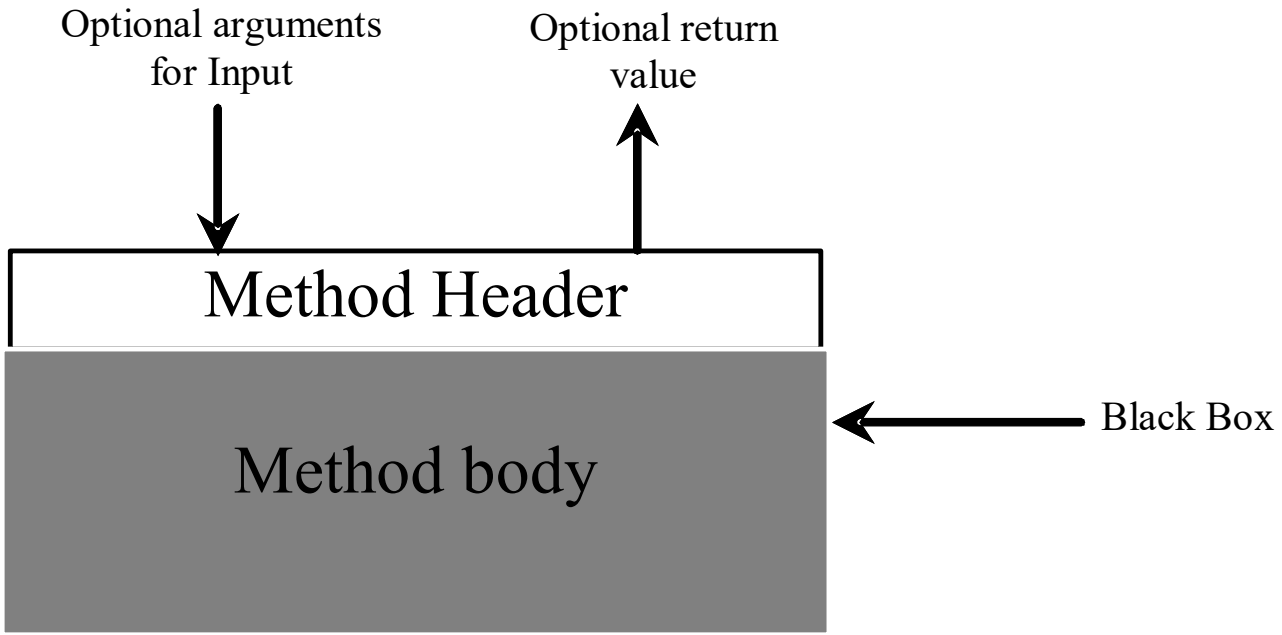
- To distinguish between instance and static variables and methods .
- To define private data fields with appropriate, get and set methods .
- To encapsulate data fields to make classes easy to maintain
- To define the scope and visibility of the data fields and class members in the code
- to restrict the levels of access to certain data, ensuring security by preventing unintended alterations.

Objectives

- How to use this reference to Refer to a class's hidden data fields
- How to use this reference to enable a constructor to call another constructor of the same class
- Method overloading ,passing &returning object and garbage collection concepts

Method Abstraction

- You can think of the method body as a black box that contains the detailed implementation for the method.

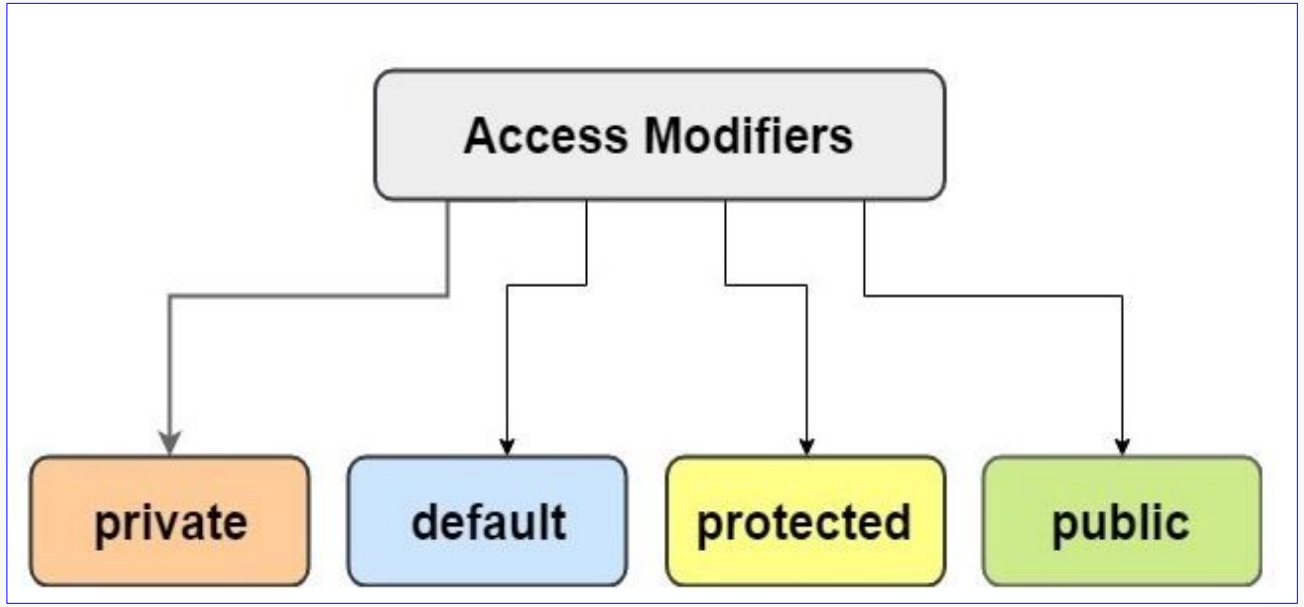


Access Specifiers (Visibility Modifiers)

- To prevent direct modifications of data fields, you should declare the fields (private). This is known as data field encapsulation.
- To provide the client the ability to access and modify private data fields, (set and get) methods should be written to change and retrieve their values, respectively.
- Why data fields should be private? To protect data and to make code easy to maintain.

Access Specifiers (Visibility Modifiers)

- **Access Modifiers**: are used to set the accessibility **(visibility)** and specify the scope of a class, constructor, variable, method, or data member.
- There are four types of access modifiers available in Java:



Access Specifiers (Visibility Modifiers)

- An access specifier is a Java keyword that indicates how a field or method can be accessed.
- **public**
  - When the public access specifier is applied to a class member, the member can be accessed by code inside the class or outside.
- **Private**
  - When the private access specifier is applied to a class member, the member cannot be accessed by code outside the class. The member can be accessed only by methods that are members of the same class.

Access Specifiers (Visibility Modifiers)

- If **public** or **private** is not used, the specifier consider **default**.
  - When the **default** access specifier is applied to a class member, The member can be accessed by any class in the same package (package-private). The default modifier restricts access to within a package.
- **Protected**
  - used with inheritance. **(will be discussed later).**

Accessor (getters) and Mutator (setters) Methods

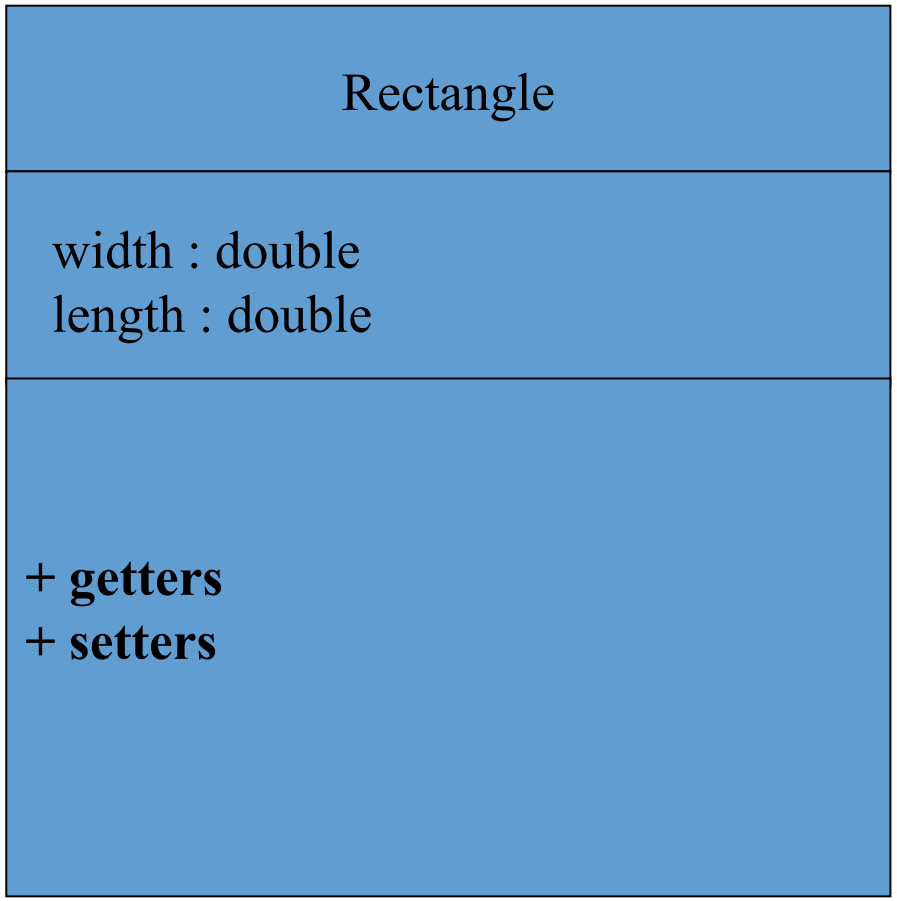
- Because of the concept of data hiding, fields in a class are private.
- The methods that retrieve the data of fields are called ***accessors(setters).***
- The methods that modify the data of fields are called ***mutators(getters).***
- Each field that the programmer wishes to be viewed by other classes needs an accessor.
- Each field that the programmer wishes to be modified by other classes needs a mutator.

Accessors and Mutators

Class name goes here

Fields are listed here

Accessors & Mutators



Accessors and Mutators

- For the Rectangle example, the accessors and mutators are:
  - setLength: Sets the value of the length field.
  - public void setLength(double len) …
  - setWidth: Sets the value of the width field.
  - public void setLength(double w) …
  - getLength: Returns the value of the length field.
  - public double getLength() …
  - getWidth: Returns the value of the width field.
  - public double getWidth() …
- Other names for these methods are ***getters*** and ***setters.***

Setter & getter Methods

- In view of the foregoing, **private** variables can only be accessed within the same class. However, it is possible to access them if we use public ***get*** and ***set*** methods.
- Getter and Setter are methods used to **protect** your data and make your code more secure.
- **Getter and Setter** make the programmer convenient in setting and getting the value for a particular data type.
- The syntax for both is that they start with either get or set, followed by the name of the variable, with the first letter in an upper-case example: **setName()** and **getName()**, to **change** and **retrieve** the value of the name data field, respectively.

Getter method: syntax

- The getter method starts with the word "**get**" followed by the variable name, getter method returns the value in different data types; int, String, double, float, etc.
- Below is an example of the syntax of a getter method:

```
public returnType getPropertyName (){
return property;
}
```

Getter method: Example

Circle.java | NewMain.java

```java
public class Circle {

    private double radius;
    private String name;
    private double dimension;
    private String color;

    public double getRadius() {
        return radius;
    }
```

The **getRadius()** method **returns** the value of the variable name (radius).

Setter method: syntax

- Setter method sets or updates the value of any given. It starts with the word “set” followed by the variable name.
- Below is an example of the syntax of a setter method:

```
public  void setPropertyName (dataType newPropertValue){
this.property = newPropertValue;
}
```

Setter method: Example

Circle.java | NewMain.java

```java
public class Circle {

    private double radius;
    private String name;
    private double dimension;
    private String color;

    public void setRadius(double radius) {
        this.radius = radius;
    }
```

The **setRadius(dataType newValue)** method receives the parameter (new **radius** value) and **assigns** it to the variable (radius), **this** keyword is used to refer to the current object.

The other purpose of the set methods is to check the validity of the data fields values.

Setter method: Validation Example

Circle.java | NewMain.java

```java
public class Circle {

    private double radius;
    private String name;
    private double dimension;
    private String color;

    public void setRadius(double radius) {
        if(radius>0){
        this.radius = radius;}
        else{
            System.out.println("invalid input");
        }
    }
```

In the example before, the values of the **radius** can't be negative. This condition is checked in the set method, which **should be invoked using the constructors** to ensure that the data fields are valid even during the creation of objects.

Setter method: Validation Example

```java
public Circle() {
    setRadius(radius);
}
```

```java
public void setRadius(double radius) {
    if (radius > 0) {
        this.radius = radius;
    } else {
        System.out.println("invalid input");}
    }
```

```java
public Circle(double radius, String color) {
    setRadius(radius);
    this.color = color;
}
```

Cont. Setter method: Validation Example

- Note! Remember: The ***set*** method that is checking the validity of the data fields **should be invoked through the constructors** to make sure that the data fields are valid even during the creation of objects.

Getter & Setter methods: Example

```java
public class Circle {
    private double radius;
    private String name;
    private double dimension;
    private String color;

    public Circle(double radius, String name, double dimension, String color) {
        setRadius(radius);
        setName(name);
        setDimension(dimension);
        setColor(color);
    }

    public void setRadius(double radius) {
        if (radius > 0) {
            this.radius = radius;
        } else {
            System.out.println("invalid input");
        }
    }

    public void setColor(String color) {
        if (color.length() != 0) {
            this.color = color;
        }
    }
```

Cont. Getter & Setter methods: Example

```
public void setDimension(double dimension) {
    if (dimension > 0) {
        this.dimension = dimension;
    }
}

public void setName(String name) {
    if (name != null) {
        this.name = name;
    }
}

public double getRadius() {
    return radius;
}

public String getColor() {
    return color;
}

public double getDimension() {
    return dimension;
}

public String getName() {
    return name;
}
```

Cont. Getter & Setter methods: Example

```
    public double area() {
        return Math.pow(radius, 2) * 3.14;


    }


    @Override
    public String toString() {
        return "Circle(" + "radius=" + radius + ", name=" + name +
                ", dimension=" + dimension + ", color=" + color + '}';
    }
}
//End of Circle class
```

Getter & Setter method: (Main Class)

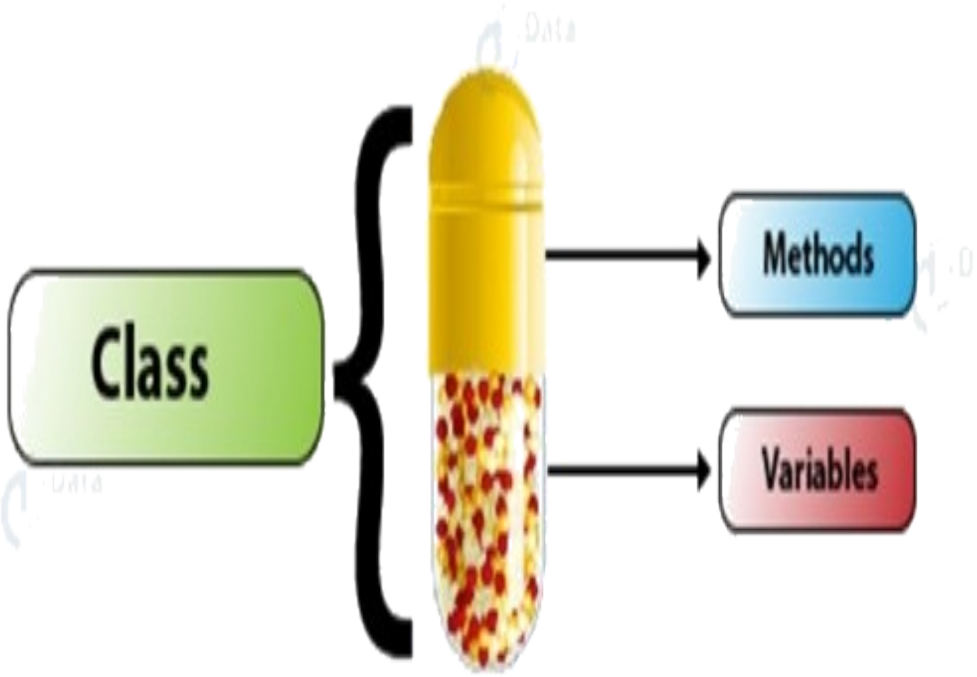
```java
public class NewMain {

    public static void main(String[] args) {
        Circle obj1 = new Circle(12.5, "football", 3, "Green");
        Circle obj2 = new Circle(3, "wall watch", 2, "Green");
        obj2.setRadius(-1);
        obj1.setColor("Red");
        String report = obj1.toString();
        String report2 = obj2.toString();
        System.out.println(report + "\n" + report2);

    }
}
```

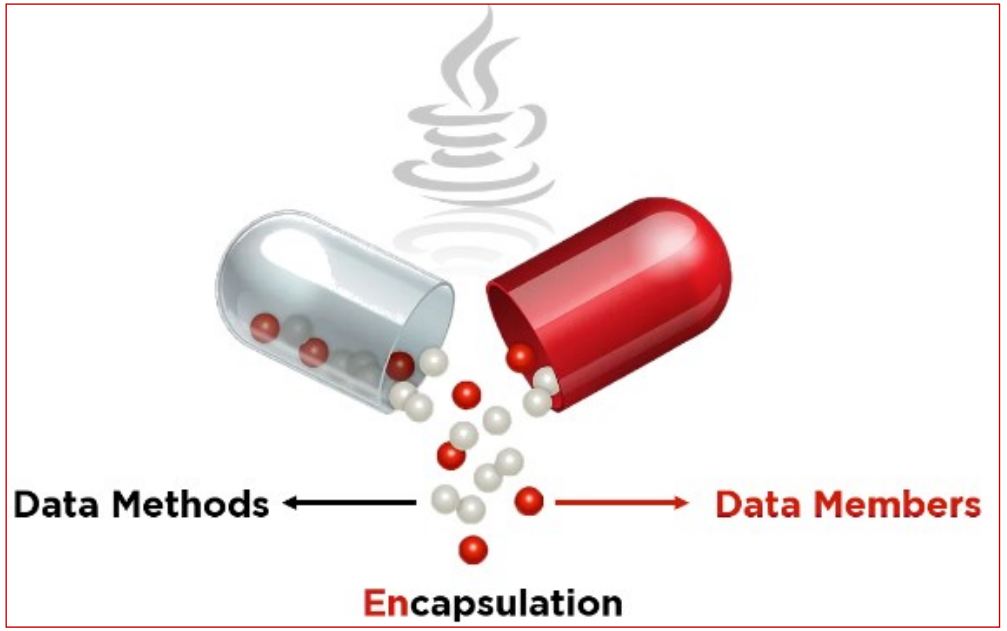
Output - Circle (run)

```
run:
invalid input
Circle{radius=12.5, name=football, dimension=3.0, color=Red  }
Circle{radius=3.0, name=wall watch, dimension=2.0, color=Green}
BUILD SUCCESSFUL (total time: 2 seconds)
```

Encapsulation

- Encapsulation: is defined as binding together the data and the methods that manipulate them.





Data Hiding

- An object hides its internal, private fields from code that is outside the class that the object is an instance of.
- Only the class's methods may directly access and make changes to the object’s internal data.
- Code outside the class must use the class's public methods to operate on an object's private fields.
- Data hiding is important because classes are typically used as components in large software systems, involving a team of programmers.
- Data hiding helps enforce the integrity of an object's internal data.

Data fields Encapsulation (Attributes & Constructors)

```java
public class Circle {

    private double radius;
    private String name;
    private double dimension;
    private String color;

    public Circle() {
    }

    public Circle(double radius, String name, double dimension, String color) {
        this.radius = radius;
        this.name = name;
        this.dimension = dimension;
        this.color = color;
    }

    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    @Override
    public String toString() {
        return "Circle{" + "radius=" + radius + ", name=" + name
                + ", dimension=" + dimension + ", color=" + color + '}';
    }
}
```

```java
public class NewMain {

    public static void main(String[] args) {
        Circle c1 = new Circle();
        c1.radius = 25;
        System.out.println(c1.radius);
        c1.name = "Circle shape";
        System.out.println(c1.name);
        c1.dimension = 1;
        System.out.println(c1.dimension);
        c1.color = "red";
        System.out.println(c1.color);
    }
}
```

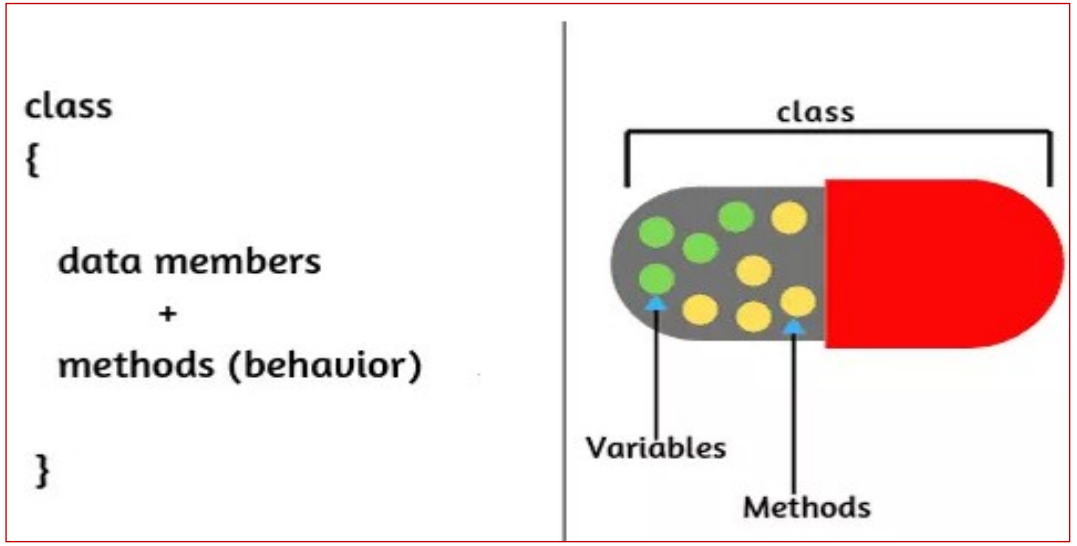
Circle class has **private** data members. In the **NewMain** class, we are going to access these private members from outside the class. Hence, there are compile-time errors **Why?**)

Data fields Encapsulation

- **The cause of the compile-time errors in the previous example from line 6 to 13 in the class (NewMain)** that, the radius, name dimension, and color are defined private and cannot be accessed from outside the **Circle** class.
- To access and modify these private data fields, **the (setter and getter) methods** should be used **to change and retrieve their values.**

Why Encapsulation?

- Better control of class attributes and methods.
- Class attributes can be only read-only (if you only use the get method) or write-only (if you only use the set method).
- Flexibility: the programmer can change one part of the code without affecting other parts.
- Increased security of data.



Exercises

- Write a Java program to create a class called Person with private instance variables name, age. and country. Provide public getter and setter methods to access and modify these variables.

- Write a Java program to create a class called BankAccount with private instance variables accountNumber and balance. Provide public getter and setter methods to access and modify these variables.

Exercises

- Write a Java program to create a class called Car with the following class members:
  - private instance variables company_name , model_name of type String , year, and mileage of type integer.
  - Provide public getter and setter methods to access and modify the variables.
  - The value of year can't be greater than current year.
  - The model name can't be empty String.
  - three different constructors.

Static Method

- static method: a **method that belongs to a class**, **not** a specific object.
- **A static method can be invoked without the need for creating an instance of a class .**

Static Method

- Static methods (class methods) are not tied to a specific object.
  - A static method can be invoked without the need for creating an instance of a class.
  - A static method can access static data member and can change the value of it.
  - The static method can not use non static data member or call non-static method directly.
  - They are typically used to create utility classes, such as the **Math** class in the Java Standard Library.
  - **this** and **super** cannot be used in static context *(explained later).*

Static Method

- Examples of static methods in java:
  - `main()` `System.out.println()`
  - All the methods defined in the **Math** class are static methods ex: `Math.sqrt()`.

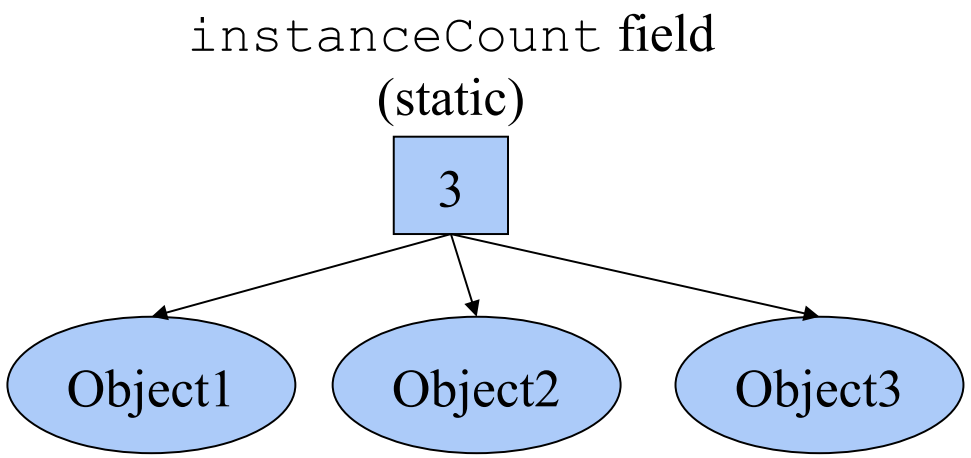
```
public class MyClass {
  public static void main(String[] args) {
    double num, root;
    …
    root = Math.sqrt(num);
    System.out.println("Root is " + root);
  } …
```

Static Variables, Constants, and Methods

- To declare static variables, constants, and methods, use the static modifier.
- Static constants are final variables shared by all the instances of the class.
- Static variables ***(class variables)*** are shared by all the instances of the class.
  - The static variable gets memory only once in the class area at the time of class loading.
  - Static data m.embers of class exist even when no object of class type is instantiated

Static Variables

- `private static int instanceCount = 0;`
  - The field is initialized to 0 only once, regardless of the number of times the class is instantiated.
  - Primitive static fields are initialized to 0 if no initialization is performed.



Static Data Field

- The static keyword is a non-access modifier used for attributes, which makes their values the same for all the objects; i.e., when we declare a field static, exactly a single copy of that field is created and shared among all instances of that class.
- The static keyword is mainly used for memory management.
- The value of the (static) field can be changed like any other data fields.

Static Data Field

- The method that returns the (static) field should be defined as (static) as well.
- The method that Modify the (static) field should be defined as (static) as well.
- Since the value (static) field is the same for all the objects, you can invoke the (get) method that returns the value of the (static) field either via an object name or via the class name itself.

Static Data Field: Explanation

```
1
2   public class Circle {
3
4       private final double radius;
5       private final String name = "Circle";
6       private static double dimension;
7       private String color;
```

In the above example, we want to make the value of **dimension** data field is the same for all the objects. So, as shown in line 6 of a class circle, we add a **static** keyword for the **dimension** data members.

Static Data Field: at object level Explanation

```java
public class Circle {
    private final double radius;
    private final String name = "Circle";
    private static double dimension;
    private String color;

    public Circle(double radius, double dimension, String color) {
        this.radius = radius;
        setDimension(dimension);
    }

    public Circle(double radius) {
        this.radius = radius;
        setDimension(dimension);    }

    public Circle() {
        this(1.5);
        setDimension(12); }

    public static void setDimension(double dimension) {
        if (dimension > 0) {
            Circle.dimension = dimension;
        }
    }

    public static double getDimension() {
        return dimension;    }
```

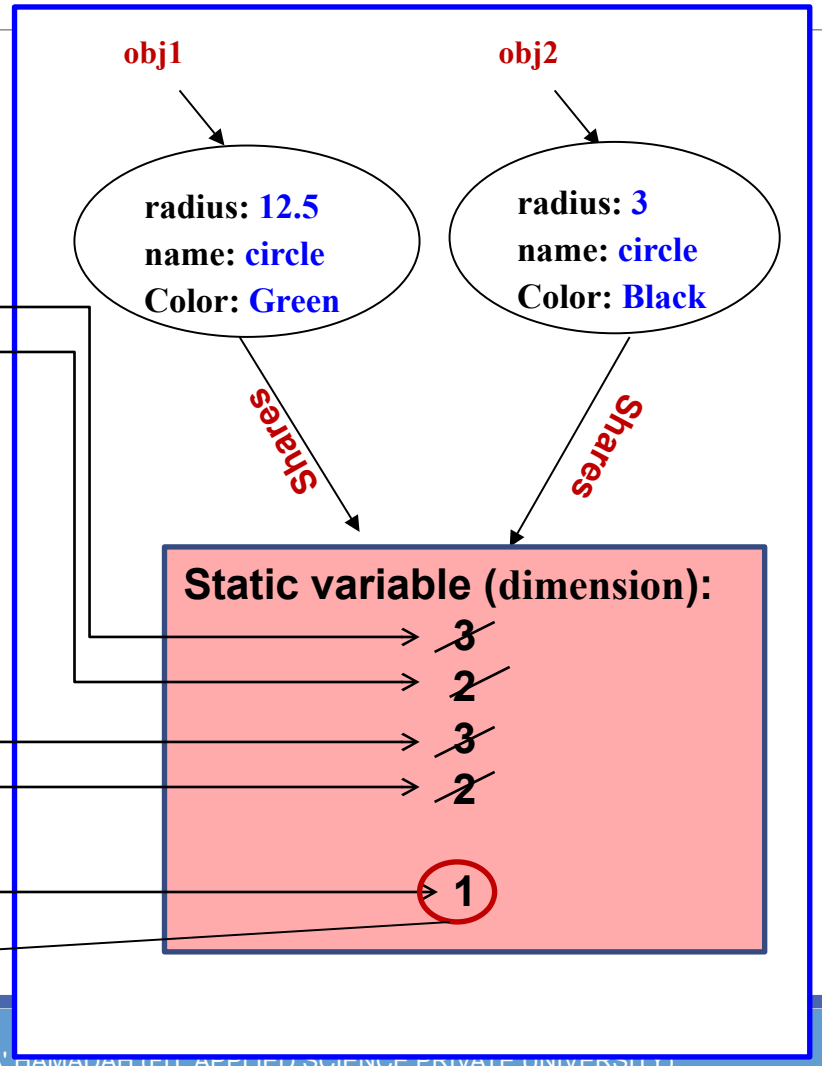
Remember:

The method that returns or Modify the (**static**) field should be defined as (**static**) as well

Static Data Field: Main

Circle.java | NewMain.java

```
1   public class NewMain {
2
3       public static void main(String[] args) {
4           Circle obj1 = new Circle(12.5, 3, "Green");
5           Circle obj2 = new Circle(3, 2, "Black");
            obj1.setDimension(3);
            obj2.setDimension(2);
            obj1.setDimension(1);
9           Circle.getDimension();
10          String report = obj1.toString();
11          String report2 = obj2.toString();
12          System.out.println(report + "\n" + report2);
13
14      }
15  }
```



The value of (1) is the last value that will be stored in the memory for the variable (dimension).

Static Data Field: Main

Circle.java | NewMain.java

```java
public class NewMain {

    public static void main(String[] args) {
        Circle obj1 = new Circle(12.5,3, "Green");
        Circle obj2 = new Circle(3, 2, "Black");
        obj1.setDimension(3);
        obj2.setDimension(2);
        obj1.setDimension(1);
        Circle.getDimension();
        String report = obj1.toString();
        String report2 = obj2.toString();
        System.out.println(report + "\n" + report2);

    }
}
```

Since the value (static) field is the same for all the objects, you can invoke the (get or set) method that returns or modify the value of the (static) field either via an object name or via the class name itself.

Static Data Field: Main

Circle.java | NewMain.java

```java
public class NewMain {

    public static void main(String[] args) {
        Circle obj1 = new Circle(12.5,3, "Green");
        Circle obj2 = new Circle(3, 2, "Black");
        obj1.setDimension(3);
        obj2.setDimension(2);
        obj1.setDimension(1);
        Circle.getDimension();
        String report = obj1.toString();
        String report2 = obj2.toString();
        System.out.println(report + "\n" + report2);

    }
}
```

Output:

```
run:
Circle{radius=12.5, name=Circle, dimension=1.0, color=Green}
Circle{radius=3.0  name=Circle, dimension=1.0, color=Black}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Method Overloading

- Method overloading: More than one method can have the same name.
- Two methods are said to have **different formal parameter** lists:
  - If both methods have a different number of formal parameters.
  - If the number of formal parameters is the same in both methods, the data type of the formal parameters in the order you list must differ in at least one position.

Method Overloading, cont.

- public void methodOne(int x)
- public void methodTwo(int x, double y)
- public void methodThree(double y, int x)
- public int methodFour(char ch, int x, double y)
- public int methodFive(char ch, int x, String name)

These methods all have different formal parameter lists.

Method Overloading, cont.

- public void methodSix(int x, double y, char ch)
- public void methodSeven(int one, double u, char firstCh)
- The methods **methodSix** and **methodSeven** both have three formal parameters, and the data type of the corresponding parameters is the same.

These methods all have the same formal parameter lists.

Method Overloading, cont.

- Method overloading: Creating several methods **within a class** with the same name.
- The **signature** of a method consists of the method name and its formal parameter list. Two methods have different signatures if they have either different names or different formal parameter lists. **(Note that the signature of a method does not include the return type of the method.)**

Method Overloading, cont.

- The following method headings correctly overload the method **methodXYZ**:
- public void methodXYZ()
- public void methodXYZ(int x, double y)
- public void methodXYZ(double one, int y)
- public void methodXYZ(int x, double y, char ch)

Method Overloading, cont.

- The following method headings correctly overload the method **add()**:

```
public int add(int num1, int num2) {
   int sum = num1 + num2;
         return sum; }
public String add (String str1, String str2) {
   String combined = str1 + str2;
         return combined; }
```

Method Overloading, cont.

public void methodABC(int x, double y)

public int methodABC(int x, double y)

- Both these method headings have the same name and same formal parameter list.
- These method headings to overload the method **methodABC** are incorrect.
- In this case, the compiler will generate a syntax error. **(Notice that the return types of these method headings are different.)**

Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match. This is referred to as ***ambiguous invocation***.
- Ambiguous invocation is a compile error.

Ambiguous Invocation, cont.

```
public class AmbiguousOverloading {
  public static void main(String[] args) {
    System.out.println(max(1, 2));  }
  public static double max(int num1, double num2) {
    if (num1 > num2)
      return num1;
    else
      return num2; }
```

```
public static double max(double num1, int num2) {
    if (num1 > num2)
      return num1;
    else
      return num2;
  }
}
```

The (this) reference

The **(this)** keyword is the name of a reference that reference to a calling object itself. There are several uses of this operator:

- Reference a class's hidden data fields: **Example:**

Circle.java | test.java

```java
public class Circle {

    double radius;
    String color;

//Constructors:
    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }
public Circle(double radius) {
        this.radius = radius;
    }

}//End of class
```

The (this) reference: Explanation

```
public Circle(double radius, String color) {
    this.radius = radius;
    this.color = color;
}
```

- In this case, the parameters of the (Circle) Constructor have the same name as the data fields (radius, and color). **To distinguish** between the parameter name (radius, and color) and the data field (radius, and color), the (this) operator should be used to access the data field.
- **But** if the parameters of the (Circle) Constructor have the same name as the data fields (radius, and color) and (this) operator is not used, then in this case it will be dealt with (data field (radius, and color)) **as local variable**.

The (this) reference:

- Enable a constructor to call another constructor of the same class: **Example:**

Circle.java | test.java

```java

public class Circle {

    double radius;
    String color;

//Constructors:
    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    public Circle(double radius) {
        this(radius, "red");
    }

    public Circle() {
        this(10, "Black");
    }
}//End of class
```

The (this) reference:

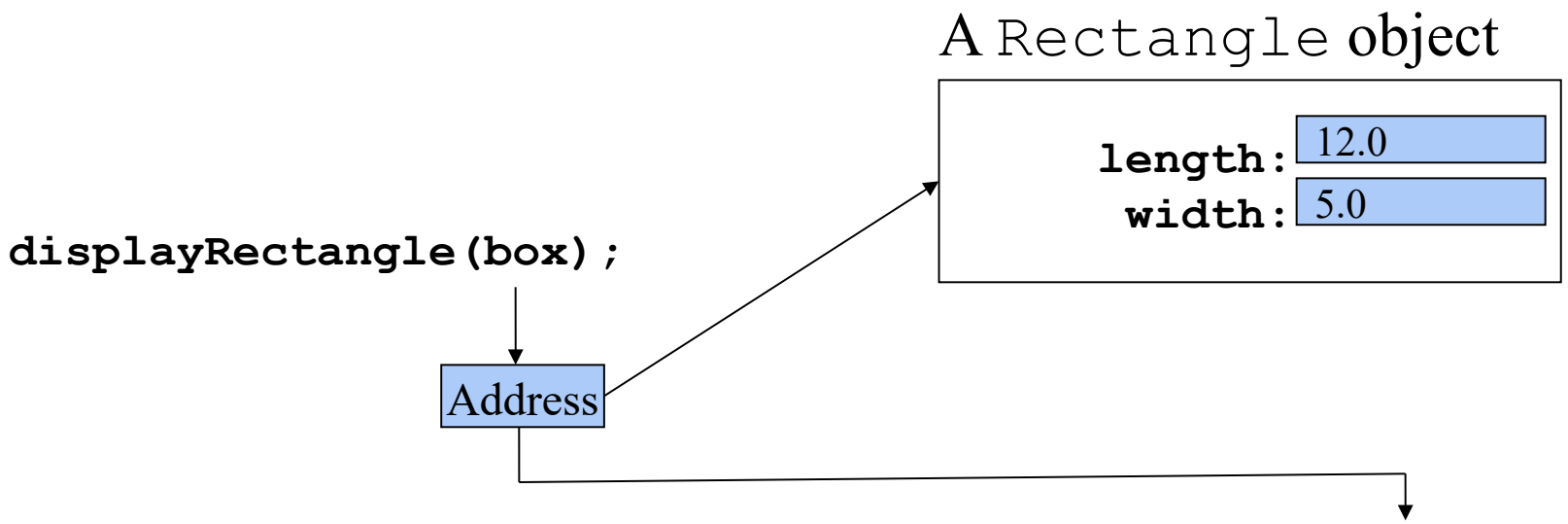
```
8      public Circle(double radius, String color) {
9          this.radius = radius;
10         this.color = color;
11     }
12
13     public Circle(double radius) {
14         this(radius, "red");
15     }
```

- In the above example, line 14 invokes the constructor with 2 parameters which is in line 8 to set the data fields values as passed from line 14 (i.e., the value of (radius), red)

Passing Objects as Arguments

- Objects can be passed to methods as arguments.
- Java passes all arguments by value.
- When an object is passed as an argument, the value of the reference variable is passed.
- The value of the reference variable is an address or reference to the object in memory.
- A ***copy*** of the object is not passed, just a pointer (a reference) to the object.
- When a method receives a reference variable as an argument, it is possible for the method to modify the contents of the object referenced by the variable.

Passing Objects as Arguments




```
public static void displayRectangle(Rectangle r)
{
  // Display the length and width.
  System.out.println("Length: " + r.getLength() +
            " Width: " + r.getWidth());
}
```

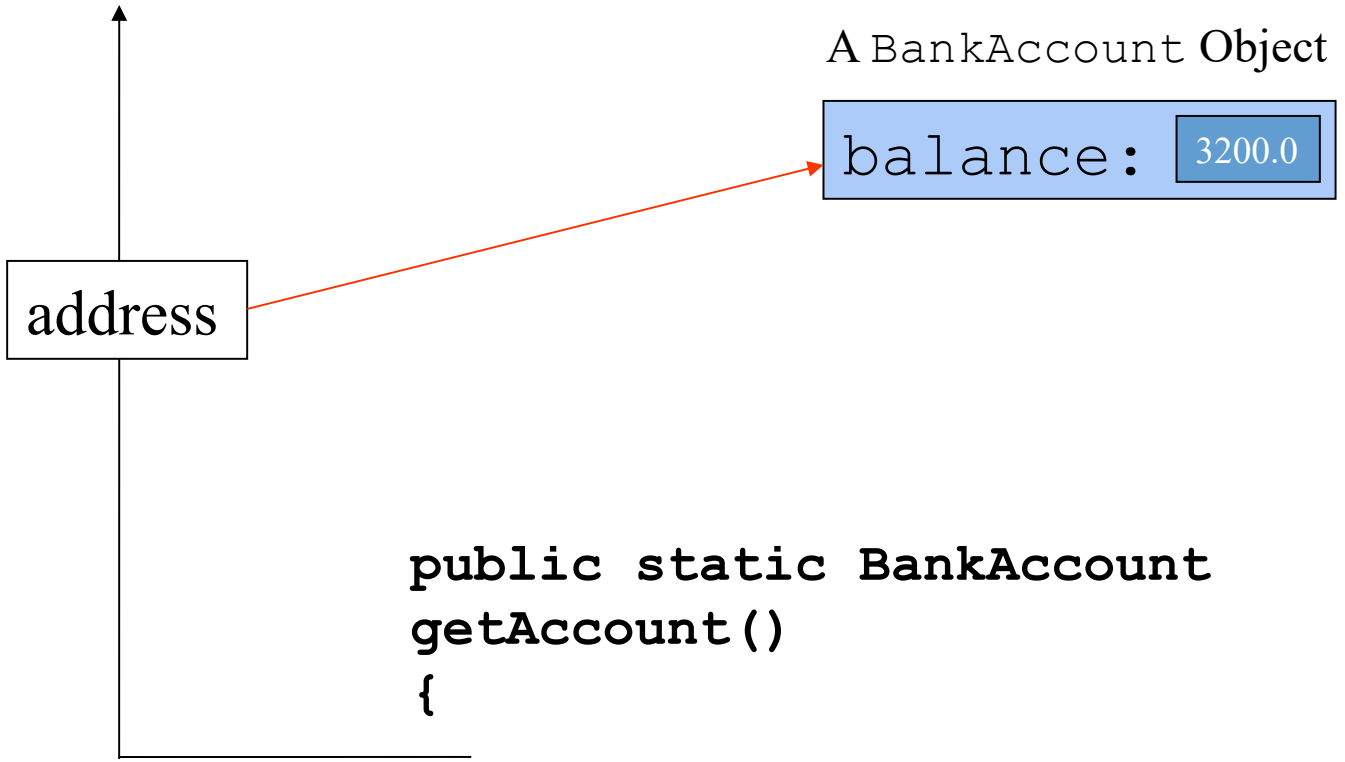
Returning Objects From Methods

- Methods are not limited to returning the primitive data types.
- Methods can return references to objects as well.
- Just as with passing arguments, a copy of the object is **not** returned, only its address.
- Method return type:

```
public static BankAccount getAccount(){
        …
return new BankAccount(balance);}
```

Returning Objects From Methods

```
account = getAccount();
```




```
public static BankAccount
getAccount()
{
    …
    return new BankAccount(balance);
}
```

Using null Reference

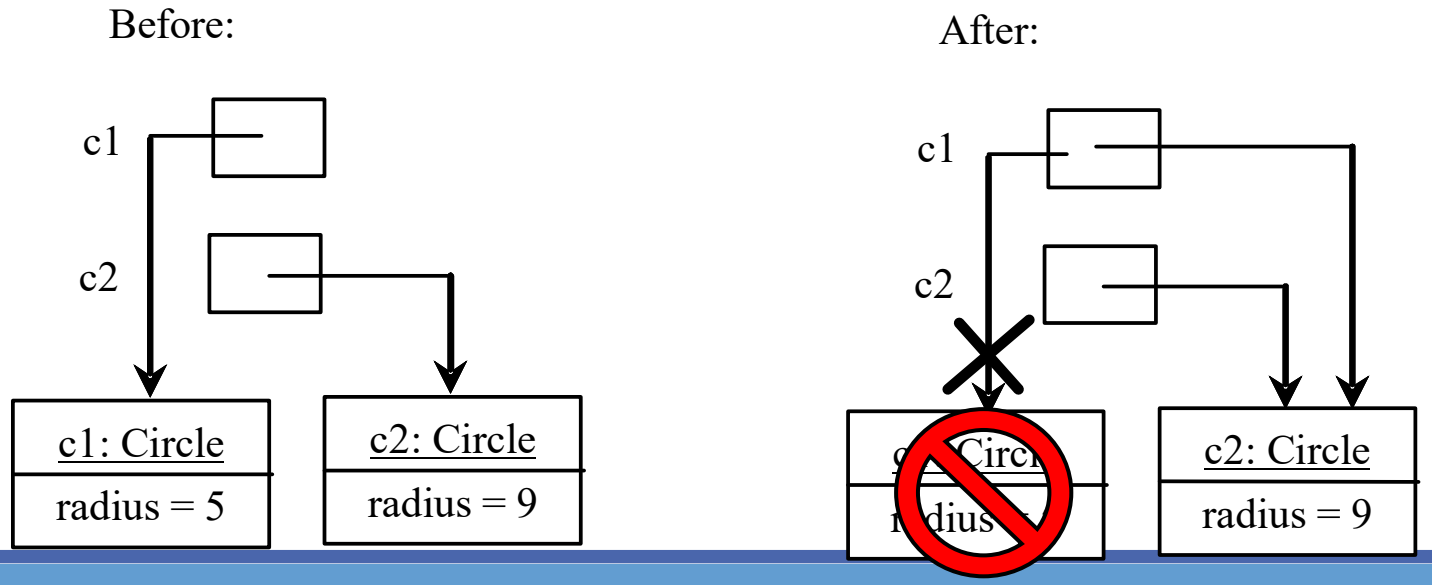
- A null reference is a reference variable that points to nothing.
- A special null value may be assigned to an object reference, but not to a primitive.
- You can compare object references to null.
- You can remove the association to an object by setting the object reference to null.

```
Car c1;                    //Declare object reference
…
if (c1 == null)        //Ref not initialized?
   c1 = new Car();     //Create a Car object
…
c1 = null;             //Forget the Car object
```

Garbage Collection

- As shown in the following figure, after the assignment statement c1 = c2, c1 points to the same object referenced by c2. The object previously referenced by c1 is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM.

Object type assignment c1 = c2



The toString Method

- All objects have a **toString** method that returns the class name and a hash of the memory address of the object.
- We can override the default method with our own to print out more useful information
- **Overriding is one of the OOP concepts and will be discussed later in details.**

```
public String toString() {
        return "Rectangle{" + " Length: = " + Length
                    + ", Width: = " + Width "}";
                        }
```

The toString Method

- The toString method of a class can be called ***explicitly***:
  - Rectangle box = new Rectangle(10.5, 9.62);
  - System.out.println(box.toString());
- However, the **toString** method does not have to be called explicitly but is called implicitly whenever you pass an object of the class to **println** or **print**.
  - Rectangle box = new Rectangle(10.5, 9.62);
  - System.out.println(box);

The toString Method

- The **toString** method is also called implicitly whenever you concatenate an object of the class with a string.
  - `Rectangle box = new Rectangle(10.5, 9.62);`
  - `System.out.println("The box data is:\n" + box);`

(toString) method

Circle.java | test.java

```java
1   public class Circle {
2
3       double radius;
4       String color;
5
6       public Circle(double radius, String color) {
7           this.radius = radius;
8           this.color = color;
9       }
10
11      public Circle(double radius) {
12          this(radius, "red");
13      }
14
15      public Circle() {
16          this(10, "Black");
17      }
18
19      public String toString() {
20          return "radius "+ radius+ " color "+color ;
21      }
22
23  }//End of class
```

Circle.java | test.java *

```java
1
2   public class test {
3
4       public static void main(String[] args) {
5           Circle r2 = new Circle(5, "red");
6           Circle r3 = new Circle(12);
7
8           System.out.println(r2.toString());
9
10          System.out.println(r2);
11
12          System.out.println(r3.toString());
13
14          System.out.println(r3);
15
16      }
17  }
```

Output - Circle (run)

Output:

```
run:
radius 5.0 color red
radius 5.0 color red
radius 12.0 color red
radius 12.0 color red
BUILD SUCCESSFUL (total time: 0 seconds)
```

(toString) method

Circle.java | test.java

```java
public class Circle {

    double radius;
    String color;

    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    public Circle(double radius) {
        this(radius, "red");
    }

    public Circle() {
        this(10, "Black");
    }

    /* public String toString() {
        return "radius "+ radius+ " color "+color ;
    }*/

}//End of class
```

Notice that the toString method is disabled in this example

Circle.java | test.java *

```java

public class test {

    public static void main(String[] args) {
        Circle r2 = new Circle(5, "red");
        Circle r3 = new Circle(12);

        System.out.println(r2.toString());

        System.out.println(r2);

        System.out.println(r3.toString());

        System.out.println(r3);

    }
}
```

Output - Circle (run)

Output

```
run:
Circle@190d11
Circle@190d11
Circle@a90653
Circle@a90653
BUILD SUCCESSFUL (total time: 0 seconds)
```

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes :edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual ???